# ExpenseMate

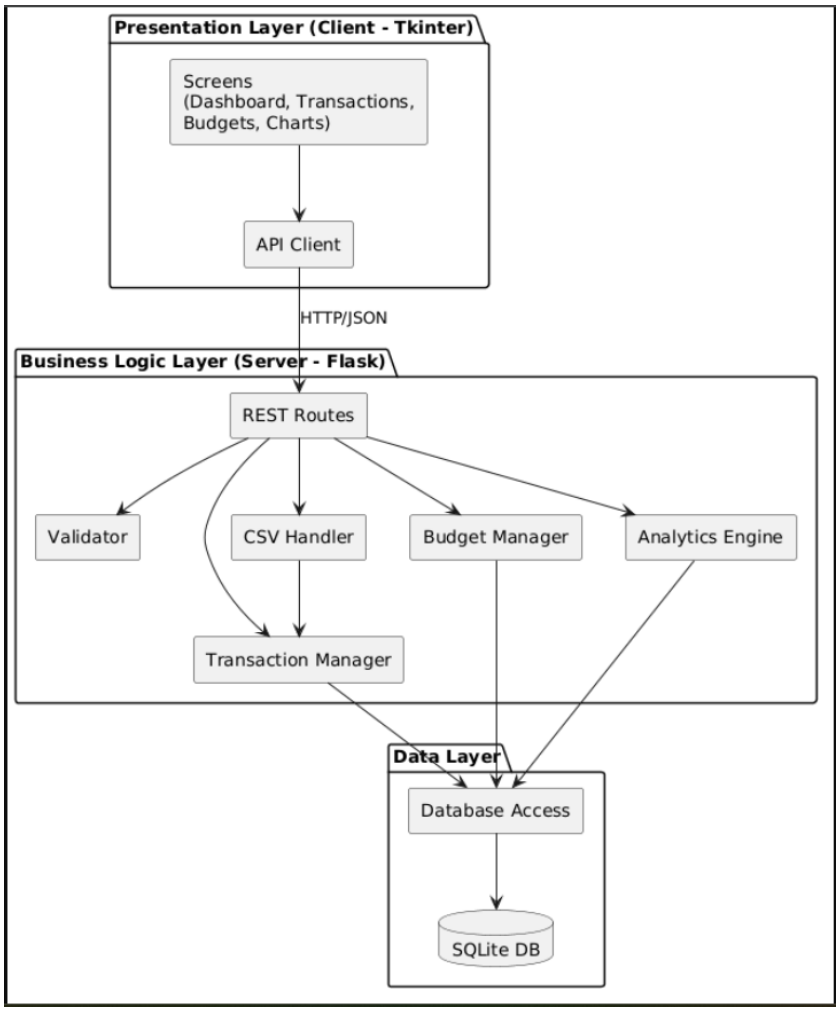
Architecture Document (Phase 2)

## 1. Architectural Style

ExpenseMate uses a Client-Server architecture running locally on one machine. The Tkinter client handles presentation, and the Flask server handles business logic and data access, communicating via HTTP/JSON on localhost. The server is internally layered (routes → business logic → data access) so each layer can change independently, which supports the maintainability requirement (NFR-05) and the Phase 5 multi-currency extension. Peer-to-Peer was rejected because the system is single-user with no need for distributed nodes.

## 2. Logical View

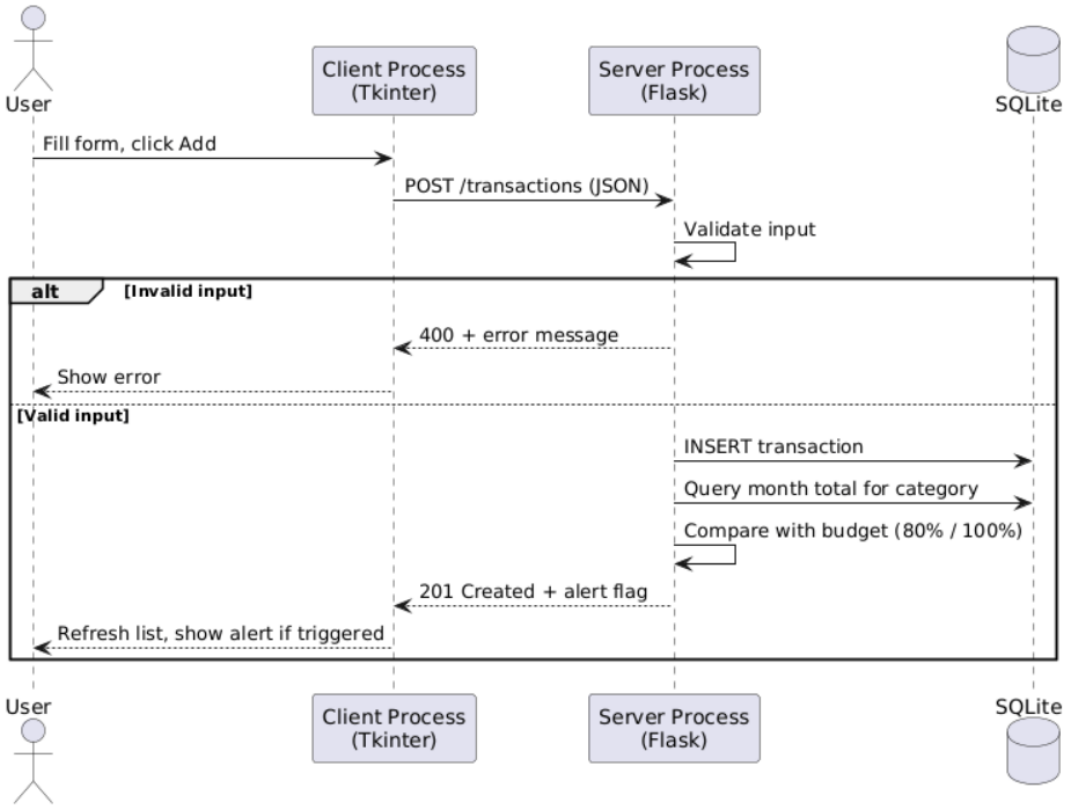
The logical view shows how the code is organised into layers: the Tkinter presentation layer talks to the Flask business logic layer over HTTP/JSON, and only the data layer touches the SQLite database.




Logical view

## 3. Process View

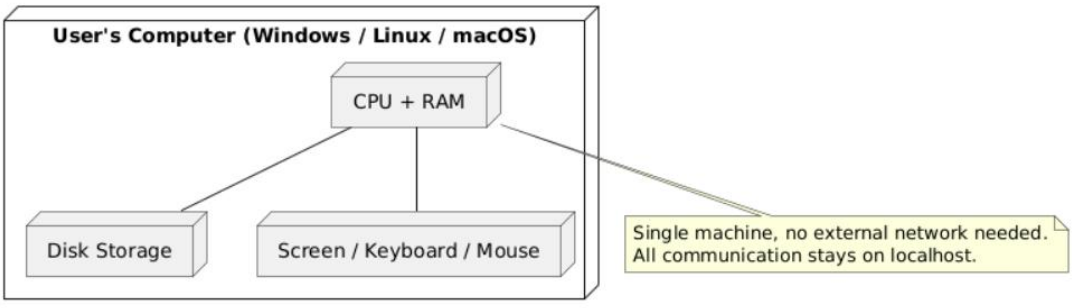
The process view shows the runtime flow for adding a transaction: the client sends the data to the server, the server validates it, saves it, checks the category budget, and returns the result together with an alert flag.




Process view (add transaction)

## 4. Physical View

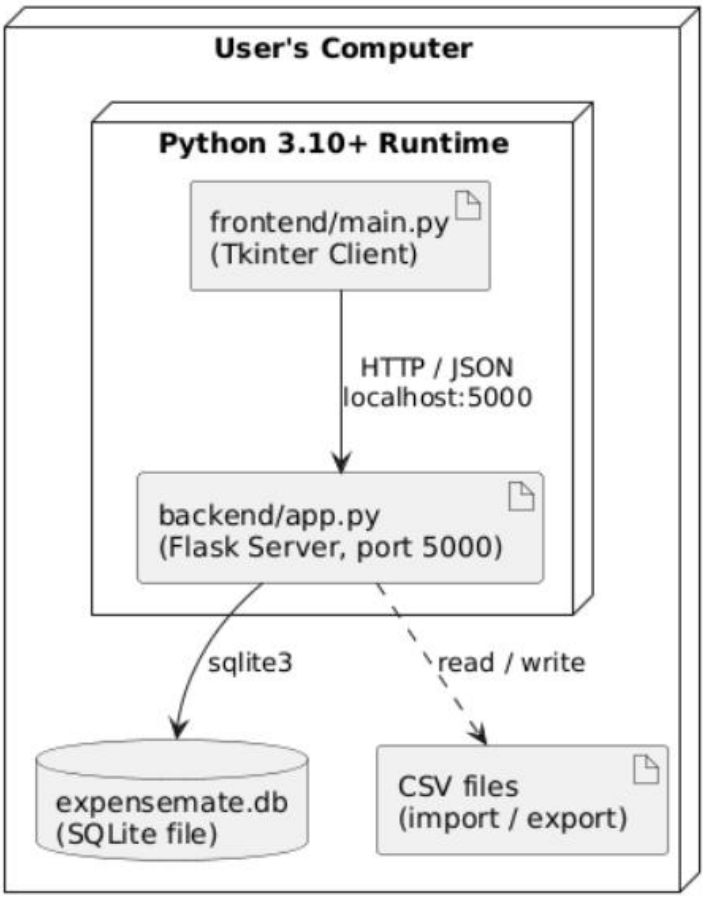
The physical view shows the hardware: everything runs on a single user machine with no external network, and all communication stays on localhost.




Physical view

## 5. Deployment View

The deployment view shows which software runs where: the client and server run inside one Python runtime, the server reads and writes the SQLite file, and CSV files are used for import and export.




Deployment view

## 6. Coding Standards

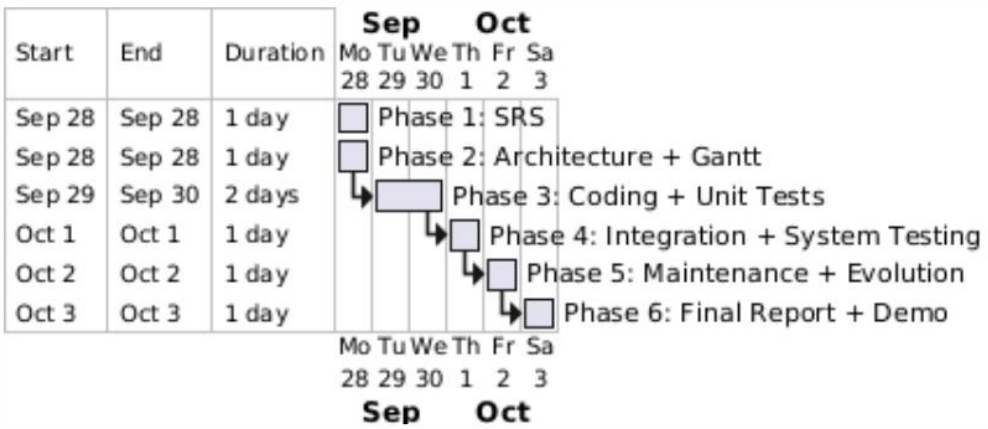
| Area | Standard |
|---|---|
| **Style** | PEP 8, max line length 100 |
| **Naming** | snake_case functions/variables, PascalCase classes, UPPER_CASE constants |
| **Docs** | One-line docstring on every function |
| **Structure** | UI code never touches the DB directly; only the server does |
| **Git** | Commit per feature, messages like feat: add budget alert or fix: csv date parsing |
| **Complexity** | Max cyclomatic complexity of 10 per function (NFR-05) |

## 7. Tools

| Purpose | Tool |
|---|---|
| **Language** | Python 3.10+ |
| **Backend** | Flask, SQLite (sqlite3) |
| **Frontend** | Tkinter, matplotlib |
| **CSV** | pandas |
| **Testing** | pytest |
| **Metrics** | radon |
| **Version control** | Git + GitHub |
| **Diagrams** | PlantUML |
| **Editor** | VS Code |

## 8. Project Timeline (Gantt Chart)

The timeline below shows the six phases in the order they will be completed.




Gantt chart